

Programação C

Tipos de dados, estruturas de dados e tipos abstratos de dados

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

O programa de hoje

Tipos de dados

Estruturas de dados

Tipos abstratos de dados

Mãos à obra

O que é um **tipo de dados**: como interpretar os bits e o que fazer com eles



O que é uma **estrutura de dados**: como organizar muitos valores na memória



O que é um **tipo abstrato de dados**: o que se pode fazer, sem dizer como

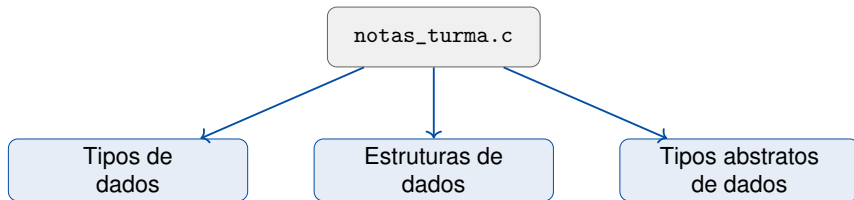
O programa de hoje

Na aula passada:

Um programa que lê 5 notas num vetor, calcula a média numa função e classifica a turma.

Hoje:

O mesmo problema cresceu: as notas ficam **cadastradas** e o programa oferece **operações** sobre a coleção.



Vamos olhar um único programa por **três lentes** diferentes.

```
1  #include <stdio.h>
2  #define CAPACIDADE 8
3
4  int inserir(double v[], int n, double nota) {
5      if (n == CAPACIDADE)
6          return n;          /* colecao cheia: ignora a nota */
7      v[n] = nota;
8      return n + 1;
9  }
10
11 double media(double v[], int n) {
12     double soma = 0.0;
13     for (int i = 0; i < n; i++)
14         soma += v[i];
15     return soma / n;
16 }
```

```
1  double maior(double v[], int n) {  
2      double m = v[0];  
3      for (int i = 1; i < n; i++)  
4          if (v[i] > m)  
5              m = v[i];  
6      return m;  
7  }
```

- Repare no padrão: toda operação recebe o **vetor** e o **contador** n — e mais nada.
- Leia com calma: você vai prever a saída do programa completo no próximo passo.

```
1  int main(void) {
2      double notas[CAPACIDADE];
3      int n = 0;
4
5      n = inserir(notas, n, 7.0);
6      n = inserir(notas, n, 8.5);
7      n = inserir(notas, n, 6.0);
8
9      printf("%d notas cadastradas\n", n);
10     printf("media: %.2f\n", media(notas, n));
11     printf("maior: %.1f\n", maior(notas, n));
12     printf("cada nota ocupa %zu bytes\n", sizeof(double));
13     printf("o vetor ocupa %zu bytes\n", sizeof notas);
14     return 0;
15 }
```

Antes de executar, preveja — em dupla, 2–3 minutos

1. Escreva as **cinco linhas** de saída, exatamente como vão aparecer no terminal.
2. Que valor sai como média? Com quantas casas decimais?
3. O vetor guarda **3 notas** — quantos bytes você acha que a última linha vai informar? Por quê?

Anote as previsões: vamos compará-las com a saída real já já.

```
$ gcc notas_turma.c -o notas_turma
$ ./notas_turma
3 notas cadastradas
media: 7.17
maior: 8.5
cada nota ocupa 8 bytes
o vetor ocupa 64 bytes
```

- Compare com a sua previsão: o que bateu? O que surpreendeu?
- Duas surpresas comuns: a média 7.17 (e não 7.16?) e os 64 bytes (e não 24?).
- As respostas passam pelos três conceitos de hoje — vamos a eles.

Tipos de dados

Problema:

Para o computador, a memória é só uma sequência de **bits**. O padrão 01000001 é a letra A? O número 65? Parte de um número maior?

Os bits, sozinhos, **não dizem o que significam**.

Solução:

O **tipo de dados** responde as duas perguntas que os bits não respondem:

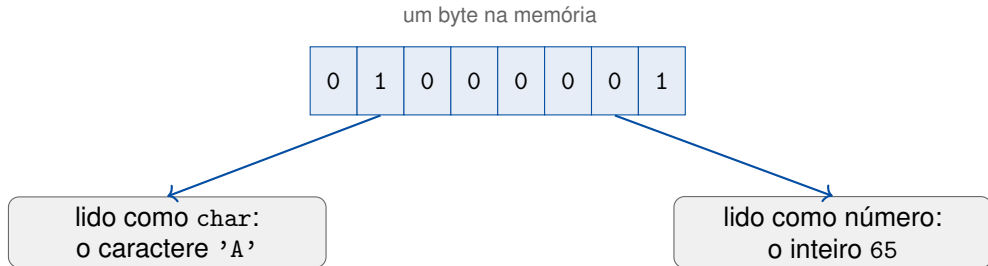
- **como interpretar** os bits;
- **o que se pode fazer** com eles.

Definição

Um **tipo de dados** é um conjunto de **valores** possíveis, as **operações** permitidas sobre eles e a **representação** desses valores na memória (quantos bytes, em que formato).

Intuição

O tipo é um **contrato de leitura**: os mesmos bits, lidos com contratos diferentes, significam coisas diferentes — como uma sequência de letras que muda de sentido conforme o idioma em que você a lê.



- O tipo também decide a **operação**: $7 / 2$ vale 3 (divisão inteira), mas $7.0 / 2$ vale 3.5.
- Na aula passada: trocar `soma` para `int` mudou a média — mesmo programa, outro contrato de leitura.

Tipo	Guarda	Tamanho típico	Exemplo de literal
char	um caractere	1 byte	'A'
int	inteiro	4 bytes	42
float	real, ~7 dígitos	4 bytes	3.14f
double	real, ~15 dígitos	8 bytes	3.14

- O operador `sizeof` informa quantos **bytes** um tipo (ou uma variável) ocupa — o resultado tem tipo `size_t` e é impresso com `%zu`.
- “Típico” porque o padrão C fixa **mínimos**, não valores exatos: os tamanhos podem variar entre máquinas.

- cada nota ocupa 8 bytes — é o `sizeof(double)`: a **representação** faz parte do tipo.
- A média exata é $21,5/3 = 7,1\bar{6}$; o `double` guarda o valor com ~ 15 dígitos e o `%.2f` **arredonda** para 7.17 — não trunca.
- Se soma fosse `int`, a divisão `soma / n` seria inteira: média 7.00. O **tipo escolhe a operação**.
- `n` é `int` porque conta coisas; notas guarda `double` porque notas têm casas decimais. Escolher tipo é escolher **o que o valor significa**.

O que aprendemos

- Bits não têm significado próprio; o **tipo** diz como interpretá-los.
- Tipo = **valores** + **operações** + **representação**.
- `sizeof` revela a representação; a mesma expressão muda de resultado conforme o tipo (7/2 vs. 7.0/2).

Pergunta-guia deste conceito

“Como interpreto um valor e o que posso fazer com ele?”

Estruturas de dados

Problema:

Uma variável de um tipo básico guarda **um** valor. A turma tem dezenas de notas; um texto tem milhares de caracteres.

Criar `nota1`, `nota2`, ..., `nota50` não escala.

Solução:

Uma **estrutura de dados**: uma forma de **organizar** muitos valores na memória, com **regras de acesso** conhecidas.

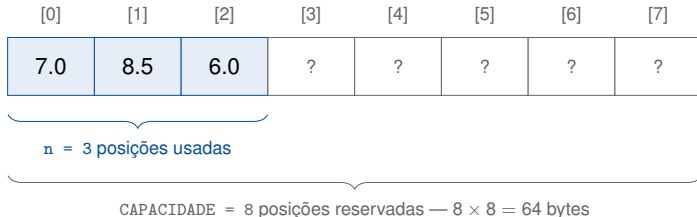
O vetor é a primeira que você já conhece.

Definição

Uma **estrutura de dados** é uma forma de organizar e relacionar dados na memória, junto com as **regras de acesso** a esses dados (onde inserir, como percorrer, como localizar).

Intuição

Uma biblioteca não é só uma pilha de livros: é a estante, a ordem nas prateleiras e o catálogo. A **organização** é o que torna o acesso rápido — os dados são os mesmos, o arranjo é que muda tudo.



- Mistério dos 64 bytes resolvido: `sizeof` mede a **capacidade reservada**, não o quanto está em uso.
- A estrutura de hoje é a **dupla** vetor + `n`: os dados, a organização (posições contíguas) e a regra (as `n` primeiras valem).
- C não verifica limites: a regra `n <= CAPACIDADE` é **nossa** responsabilidade — por isso o teste em `inserir`.

Vetor



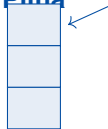
acesso direto
pelo índice

Lista encadeada



cresce e encolhe
sem capacidade fixa

Pilha



entra e sai
pelo topo

Fila



entra no fim,
sai da frente

- Cada organização favorece um uso: não existe estrutura “melhor” — existe a **adequada ao problema**.
- Neste semestre vamos **implementar** cada uma delas em C, com struct e ponteiros.

O que aprendemos

- Estrutura de dados = **organização** + **regras de acesso**, não só os dados.
- Vetor + contador n : contíguo, capacidade fixa, as n primeiras posições valem.
- Manter as regras (ex. não passar da capacidade) é responsabilidade do programa em C.

Pergunta-guia deste conceito

“Como organizo muitos valores para acessá-los do jeito que o problema pede?”

Tipos abstratos de dados

Problema:

Se quem **usa** a coleção de notas precisa conhecer o vetor, o `n` e a capacidade, qualquer mudança interna quebra todo o programa — e cada usuário pode violar as regras sem querer.

Solução:

Separar **o quê** se pode fazer (as operações e suas regras) de **como** é feito (a estrutura interna).

Essa separação tem nome: **tipo abstrato de dados** (TAD).

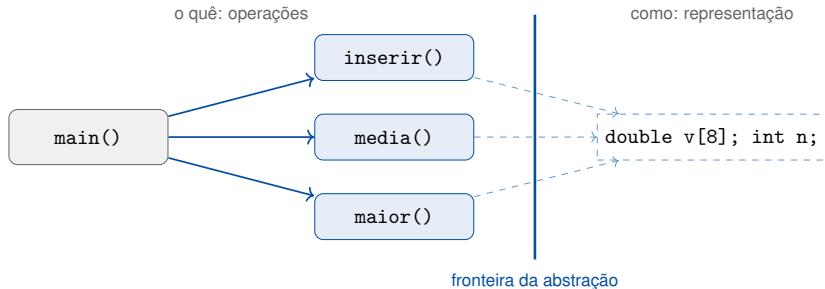
Definição

Um **tipo abstrato de dados** é a especificação de um conjunto de **operações** sobre uma coleção de valores e das **regras de comportamento** dessas operações — **sem fixar** como os dados são representados nem como as operações são implementadas.

Intuição

Um caixa eletrônico: você conhece as operações (consultar saldo, sacar, depositar) e as regras (não sacar mais que o saldo). Como o banco guarda as contas por dentro? Você não sabe — e **não precisa** saber.

A fronteira entre o quê e o como



- O `main` do nosso programa só cruza a fronteira pelas **operações** — ele nunca toca `v[]` diretamente.
- A regra “não passar da capacidade” vive **dentro** de `inserir`: quem usa não consegue violá-la.

- `printf("media: %.2f\n", m)` — você conhece **o que** ele faz; sabe **como** ele converte o `double` em texto e o entrega à tela? Não precisa.
- As funções de `math.h` (`sqrt`, `pow`): operações com comportamento conhecido, implementação escondida.
- Toda a biblioteca padrão de C é organizada assim: **contratos** no `.h`, implementação em outro lugar.

A consequência prática

Se amanhã a coleção de notas virar uma lista encadeada, só os corpos de `inserir`, `media` e `maior` mudam — o `main` continua **idêntico**. Abstração é o que permite um programa **evoluir** sem quebrar.

Conceito	Pergunta-guia	No programa de hoje
Tipo de dados	Como interpreto um valor e o que posso fazer com ele?	double, int, sizeof, arredondamento
Estrutura de dados	Como organizo muitos valores na memória?	vetor contíguo + contador n
Tipo abstrato de dados	O que se pode fazer, sem dizer como?	inserir, media, maior escondendo o vetor

Três níveis da mesma escada: **interpretar** um valor, **organizar** muitos, **esconder** a organização.

O que aprendemos

- TAD = **operações** + **regras de comportamento**, sem fixar a implementação.
- A fronteira da abstração protege as regras e isola quem usa de quem implementa.
- Em C, a fronteira se materializa em **funções** (e, adiante, em `struct` + headers).

Pergunta-guia deste conceito

“O que esta coleção faz — independente de como é feita por dentro?”

Mãos à obra

Tarefas, em ordem crescente de desafio — uma por lente

1. **Tipos:** troque `double` por `float` em todo o programa. O que muda nas duas últimas linhas da saída? E na média?
2. **Estruturas:** num laço, insira 10 notas iguais a 5.0. Quantas entram? Como você descobre, pelo programa, que as demais foram ignoradas?
3. **Abstração:** acrescente a operação `double menor(double v[], int n)` e imprima também a menor nota — sem alterar nenhuma das operações existentes.

Dica

Para cada tarefa: altere, recompile e **preveja a saída antes de executar**.

Desafio

Uma estação meteorológica registra leituras de temperatura ao longo do dia. Escreva as operações de um registro de temperaturas:

- `int registrar(double v[], int n, double t)` — aceita somente leituras entre -10 e 50°C (regra dentro da operação!);
- `double media(double v[], int n)`;
- `double amplitude(double v[], int n)` — maior menos menor leitura.

O main deve usar **apenas** essas operações.

Terminou? Estenda

Conte quantas leituras foram **rejeitadas** por estarem fora da faixa — sem quebrar a fronteira da abstração.

Os três conceitos de hoje

- **Tipo de dados:** valores + operações + representação — o contrato de leitura dos bits.
- **Estrutura de dados:** organização + regras de acesso — o arranjo dos valores na memória.
- **Tipo abstrato de dados:** operações + comportamento — o quê, sem o como.

Como trabalhamos

Prevendo, executando, investigando, modificando e criando — como será em todo o semestre.

- **Próxima aula (24/08):** registros (`struct`) — declaração, acesso, structs aninhadas, alinhamento e padding na memória. É a primeira estrutura que vamos construir de verdade.
- Fazer o tutorial guiado (Jupyter Notebook) do Classroom — ele continua as tarefas de alteração e o desafio para quem não terminou em sala.
- Ler o capítulo introdutório de Tenenbaum, Langsam & Augenstein (*Estruturas de dados usando C*) sobre tipos e abstração.
- Dúvidas: andreyoshiaki@dcomp.ufs.br ou no início da próxima aula.



André Backes.

Linguagem C: completa e descomplicada.

Elsevier, Rio de Janeiro, 2013.



Randal E. Bryant and David R. O'Hallaron.

Computer Systems: A Programmer's Perspective.

Pearson, Boston, 3 edition, 2015.

Seção 3.9: alinhamento de dados e layout de structs.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Algoritmos: teoria e prática.

Elsevier, Rio de Janeiro, 3 edition, 2012.



Paul Deitel and Harvey Deitel.

C: como programar.

Pearson Prentice Hall, São Paulo, 6 edition, 2011.



Ulrich Drepper.

What Every Programmer Should Know About Memory.

Red Hat, Inc., 2007.

Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.



Ulrich Drepper.

How to write shared libraries.

Documento técnico, 2011.

Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



Free Software Foundation.

Gnu coding standards.

Free Software Foundation.

Acesso em 2026.



Free Software Foundation.

Using the GNU compiler collection — options to request or suppress warnings.

Manual do GCC.

Acesso em 2026.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seção 6.7.2.1: structure and union specifiers.



Brian W. Kernighan and Rob Pike.

The Practice of Programming.

Addison-Wesley, Reading, MA, 1999.

Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.



Brian W. Kernighan and Dennis M. Ritchie.

C: a linguagem de programação — padrão ANSI.

Campus, Rio de Janeiro, 1989.



K. N. King.

C Programming: A Modern Approach.

W. W. Norton, New York, 2 edition, 2008.

Capítulo 16: estruturas, uniões e enumerações.



Steve McConnell.

Code Complete: A Practical Handbook of Software Construction.

Microsoft Press, Redmond, WA, 2 edition, 2004.

Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).



Robert Mecklenburg.

Managing Projects with GNU Make.

O'Reilly Media, 3 edition, 2004.



Herbert Schildt.

C completo e total.

Makron Books, São Paulo, 3 edition, 1997.



Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.

AddressSanitizer: A fast address sanity checker.

In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Ferramenta usada no tutorial para detectar vazamentos e uso após free.



Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.

Estruturas de dados usando C.

Pearson Makron Books, São Paulo, 1995.



The Linux Kernel Developers.

Linux kernel coding style.

Documentação do kernel Linux, [Documentation/process/coding-style.rst](#).

Acesso em 2026.



The LLVM Project.

Clang-format style options.

Documentação do Clang.

Acesso em 2026.



Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.

Dynamic storage allocation: A survey and critical review.

In *International Workshop on Memory Management (IWMM)*, pages 1–116. Springer, 1995.

Panorama de estratégias de alocação dinâmica e fragmentação.



Nivio Ziviani.

Projeto de algoritmos: com implementações em Pascal e C.

Cengage Learning, São Paulo, 3 edition, 2011.